

# Pricing ETL — Concurrent API Batch Pipeline

Spring Batch single-step ETL that reads formula definitions from Oracle, expands them over a 30-day lookback, and fires concurrent HTTP POST requests to a pricing valuation API — Java 11 · Spring Boot 2.3 · Spring Batch · ThreadPoolTaskExecutor · Apache HttpClient

Java 11 · Spring Boot 2.3 · Spring Batch · Spring Data JPA · ThreadPoolTaskExecutor · Java Future API · Apache HttpClient 4.5 · RestTemplate · Jackson · OpenCSV 5.7 · Oracle JDBC (ojdbc8) · Lombok

## 1 System Overview

A Spring Batch command-line ETL application that runs once on startup (CommandLineRunner). It reads pre-serialized pricing formula definitions from an Oracle staging table, expands each formula across up to 30 daily event dates, and submits each formula-date combination as a JSON POST to an external pricing valuation API using a ThreadPoolTaskExecutor (core 10, max 20 threads). Results — price, currency, UOM, status — are collected via a List<Future> and written to a CSV file. There is no scheduler, no message queue, and no retry framework — the batch executes, writes output, and exits.

### ▲ Extract Layer

DatabaseItemReader — Spring Data JPA  
Native SQL: SELECT \* FROM staging table ORDER BY formulaSetId  
Entity: UsRackPricing (formulaId, serializedFormula, validFrom/To)  
Oracle JDBC (ojdbc8) — Hikari connection pool  
Returns full List<UsRackPricing> in one read

### ► Transform & Load — Concurrent

PricingItemProcessor: formula expansion + API calls  
Group by formulaSetId; expand to 30-day date variants  
Patch eventDate per day; re-serialize → HTTP POST body  
Submit each as Callable to ThreadPoolTaskExecutor  
Collect Future<String> → parse pricing response JSON

### ▼ Write Layer

CsvItemWriter — OpenCSV 5.7  
Sort by FormulaId, then EventDate before write  
Output: target/output.csv (overwritten each run)  
Columns: FormulaId, FormulaSetVersion, Price, UOM, CCY, LastPricingDate, EventDate, Status, Details  
No quote/escape — comma-delimited plain text

### Thread Pool — ThreadPoolTaskExecutor

Core: 10 threads (always-alive). Max: 20 threads (burst capacity). Queue: LinkedBlockingQueue (unbounded). Thread name prefix: MyAPIThread-N.  
Rejection policy: AbortPolicy (throws on queue overflow).

### Pricing API — JSON over HTTP POST

Endpoint: external pricing valuation RPC service. Content-Type: application/json. LaxRedirectStrategy on HttpClient (follows redirects on POST). No explicit timeout or retry configured — single-attempt per request.

### Formula Date Expansion

For each formula, extract validFrom date. If within last 30 days (configurable: number.of.days), generate one CsvPricingModel per day from validFrom to today. Expansion ratio: up to ~30× records. Total API calls = formulaCount × matchingDays.

### Custom JSON Deserialization

CustomObjectDeserializer handles polymorphic formula components via @JsonCreator. Two concrete types: IndexPrice-based and Quantity-based. Discriminated by JSON field presence at parse time.

## 2 Concurrent API Request Lifecycle

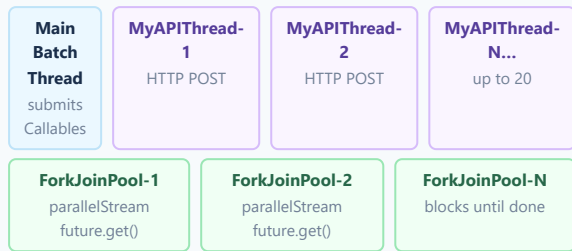
Spring Boot startup → CommandLineRunner → JobLauncher.run("myJob", emptyParams)



**Key design note:** Futures are submitted in a loop, then consumed via `parallelStream().forEach(future.get())`. Each `get()` blocks the parallel-stream thread until its assigned API call completes — effectively: producer threads (MyAPIThread-N) run HTTP calls; consumer threads (ForkJoinPool) block on results. Up to 20 API calls can be in-flight simultaneously.

### 3 Threading Architecture

#### ThreadPoolTaskExecutor — API Worker Threads



| Setting                       | Value                                    | Notes                                          |
|-------------------------------|------------------------------------------|------------------------------------------------|
| <code>corePoolSize</code>     | 10                                       | Always-alive threads (read from property)      |
| <code>maxPoolSize</code>      | 20                                       | Burst capacity under heavy load                |
| <code>queueType</code>        | <code>LinkedBlockingQueue</code>         | Unbounded — no back-pressure                   |
| <code>keepAlive</code>        | 60s                                      | Default JDK value                              |
| <code>rejectionPolicy</code>  | <code>AbortPolicy</code>                 | Throws <code>RejectedExecutionException</code> |
| <code>threadNamePrefix</code> | <code>MyAPIThread-</code>                | Visible in logs and thread dumps               |
| <b>Concurrency API</b>        | <code>java.util.concurrent.Future</code> | Legacy (not <code>CompletableFuture</code> )   |
| <b>Result collection</b>      | <code>List&lt;Future&gt;</code>          | Manual accumulation per request                |

**Concurrency model:** Java `Callable<String>` submitted per formula-date unit to the executor. Results polled via `parallelStream().forEach(f.get())` — each parallel-stream thread blocks until its future resolves. No `CompletableFuture.allOf()`, no timeout on `get()`, no circuit breaker.

### 4 Key Architecture Patterns

#### ⚡ Fork-Join via ThreadPoolTaskExecutor + Future

Main thread submits one `Callable` per formula-date to the executor (fork). A `parallelStream` over the futures list then blocks on each result (join). Up to 20 API calls execute concurrently. No `CompletableFuture` — legacy `java.util.concurrent.Future` with manual list management.

#### 📅 Time-Windowed Record Expansion

One source formula row expands to N output records by iterating daily from `validFrom` to today (bounded by `number.of.days` property). The `serializedFormulaJSON` is parsed, `eventDate` field patched in-memory, and re-serialized — reusing the pre-built request structure without regenerating it from scratch.

#### 🗄️ Staging Table as API Request Store

`serializedFormula` column in Oracle stores a pre-serialized JSON API request body. The ETL treats the DB as both a formula registry and a request cache — processors only patch the mutable date field, avoiding full request reconstruction per row.

#### 📬 Tasklet over Chunk-Oriented Step

Spring Batch step uses `Tasklet` (single `execute()` call) rather than chunk-oriented (read-process-write with commit interval). This is intentional: the entire dataset is read at once, processed in-memory with async API calls, then written in one pass — no mid-batch checkpointing needed.

#### 🔄 Polymorphic JSON Deserialization

`CustomObjectDeserializer` uses `@JsonCreator` to discriminate between two formula component types (`IndexPrice` vs `Quantity`) based on JSON field presence. Jackson resolves the concrete type at parse time — no manual type switching in business logic.

#### 🌐 Property-Driven Concurrency Tuning

`thread.core.pool.size`, `thread.max.pool.size`, `number.of.days` and `jupiter.api.url` are all externalized to `application.properties`. Thread pool and expansion window can be adjusted per environment without code change or rebuild.

## 5 Architecture Decisions

| Decision                  | Choice                                                     | Rationale                                                                                                                                                                                                                        |
|---------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Batch framework           | Spring Batch — single Tasklet step                         | Provides job execution lifecycle, status tracking, and clean CommandLineRunner entry point. Tasklet preferred over chunk because the workload is monolithic — one atomic read-expand-call-write cycle, not a streaming pipeline. |
| Concurrency API           | java.util.concurrent.Future + ThreadPoolTaskExecutor       | Straightforward submit-then-collect pattern that matches the all-at-once submission model. No reactive stream or CompletableFuture chaining needed since all futures are collected before any result is processed.               |
| HTTP client               | RestTemplate + Apache HttpClient 4.5 (LaxRedirectStrategy) | RestTemplate is the standard synchronous client in Spring Boot 2.x. LaxRedirectStrategy required because the upstream API may issue HTTP 302 on POST — default HttpClient would drop the body on redirect.                       |
| Result collection pattern | List<Future> + parallelStream().forEach(get())             | Simple and readable — all futures submitted first (maximising concurrency window), then consumed. parallelStream provides additional parallelism for JSON deserialization after network I/O completes.                           |
| Output format             | CSV via OpenCSV — file overwritten each run                | Downstream consumers expect flat CSV. Run-level idempotency: output.csv is always a complete, fresh snapshot of the latest pricing run — no append, no partitioning.                                                             |
| No retry / timeout        | Single-attempt per API call, no timeout on future.get()    | Batch is designed for a controlled internal environment where API availability is assumed. Unlimited future.get() keeps the batch alive until all responses arrive — failure propagates as exception and aborts the step.        |

## 6 Technology Stack

Java 11   Spring Boot 2.3.12   Spring Batch 4.3.x   Spring Data JPA   Spring ORM / Hibernate   ThreadPoolTaskExecutor  
java.util.concurrent.Future   java.util.concurrent.Callable   parallelStream (ForkJoinPool)   RestTemplate   Apache HttpClient 4.5.13  
HttpComponentsClientHttpRequestFactory   LaxRedirectStrategy   Jackson 2.10 (ObjectMapper)   @JsonCreator polymorphic binding  
OpenCSV 5.7.1   Oracle JDBC ojdbc8 21.1   HikariCP (default pool)   Lombok 1.18.26   Logback (rolling file appender)   SLF4J MDC